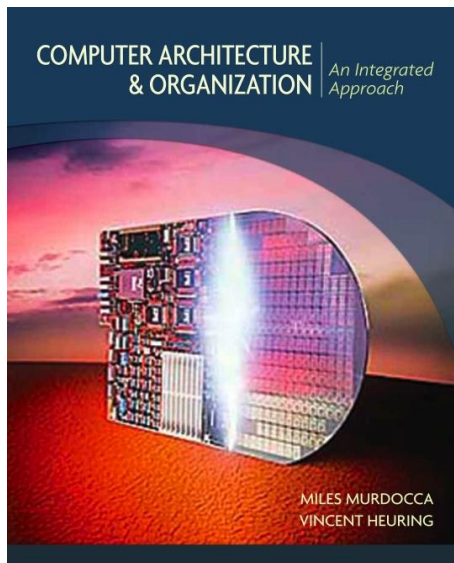


Organización de las Computadoras

Carolina Chaves Garcia



Programación directa (4) ESP32-C3

Introducción al ensamblador

Introducción al ensamblador

Archivo de ensamblador

- Contiene directivas, macros e instrucciones en lenguaje ensamblador.
- Es el archivo de entrada para el ensamblador.
- Extensiones comunes:
 - .s: archivo ensamblador tal cual.
 - .S: archivo ensamblador que será preprocesado por el preprocesador de C antes de ensamblarse.

Archivo objeto reubicable

- Contiene código objeto compilado y datos generados por el ensamblador.
- Extensión estándar: .o.
- No se puede ejecutar, sino que se utiliza como entrada para el enlazador.
- Generado por el ensamblador a partir del código en ensamblador.

Introducción al ensamblador

Formato ELF (Executable and Linkable Format)

- El formato más común de archivos objeto y ejecutables en sistemas RISC-V
- El encabezado contiene metadatos generales del archivo>
 - Indica que el archivo está en formato ELF.
 - Arquitectura del binario (RISC-V, x86, ARM, etc.).
 - Orden de bytes (little-endian en RISC-V).
 - Tipo de archivo: reubicable, ejecutable o biblioteca compartida.
 - Número y posición de los encabezados de programa y de sección.
 - Información de versión y de la ABI (Application Binary Interface) usada.
 - Opciones ABI (por ejemplo, compresión RVC o soporte de punto flotante).

Encabezado del programa

- Definen los segmentos cargables de un ejecutable o librería compartida.
- Indican tamaño, posición en el archivo, dirección de carga en memoria y permisos (lectura, escritura, ejecución).
- No están presentes en los objetos reubicables.
- Usados por el sistema operativo y el enlazador dinámico para mapear el programa en memoria.

Introducción al ensamblador

Encabezado de sección

- Describen las secciones que forman el archivo objeto o ejecutable.
- Incluyen:
 - Tamaño.
 - Compensación dentro del archivo.
 - Tipo de sección.
 - Alineación.
 - Indicadores (flags).
- Necesarios para el enlace dinámico y para las tablas de símbolos y reubicación.

Secciones más comunes

- .text es de solo lectura. Contiene código ejecutable.
- .data es de lectura y escritura. Contiene variables globales o estáticas inicializadas.
- .rodata es de solo lectura. Contiene variables constantes.
- .bss es de lectura y escritura. Contiene datos no inicializados, el sistema los inicializa en cero en tiempo de ejecución.
-

Introducción al ensamblador

Enlace de programas

- Es el proceso de fusionar varios archivos objeto reubicables (.o).
- Une las secciones equivalentes de cada archivo.
- Calcula las nuevas direcciones de memoria para los símbolos.
- Aplica las reubicaciones necesarias para que el programa funcione.

Script del enlazador

- Es un archivo de texto con extensión .ld.
- Permite controlar:
 - Dirección de carga.
 - Alineación de secciones.
 - Orden de las secciones.
- Se usa para programas embebidos o cuando se necesita ubicar código/datos en direcciones específicas de memoria.

Directivas del ensamblador

- Son instrucciones al ensamblador (no se traducen a código máquina).

Arquitectura RISC-V del ESP32-C3

Arquitectura RISC-V del ESP32-C3

Características generales

- CPU del ESP32-C3 es un RISC-V de 32 bits de un solo núcleo.
- Soporta depuración JTAG a través de la interfaz USB y un módulo de depuración (DM) integrado.
- Registros y Memoria
 - SRAM interna de 400 KB para el funcionamiento del microcontrolador. Puede configurarse como búfer de caché para instrucciones o usarse como memoria de propósito general.
 - La ROM contiene el código del gestor de arranque y bibliotecas de funciones preinstaladas.
 - NVRAM (Non-Volatile Random-Access Memory) de 16 KB para guardar datos no volátiles, como la configuración.
 -
 - Memoria Flash puede soportar hasta 16 MB de memoria flash externa, accesible a través de las cachés de instrucciones y datos en buses SPI, dual-SPI, quad-SPI y QPI.
 - Mapa de memoria: El espacio de direcciones de la arquitectura RISC-V del ESP32-C3 es más simple que el de sus predecesores Xtensa.

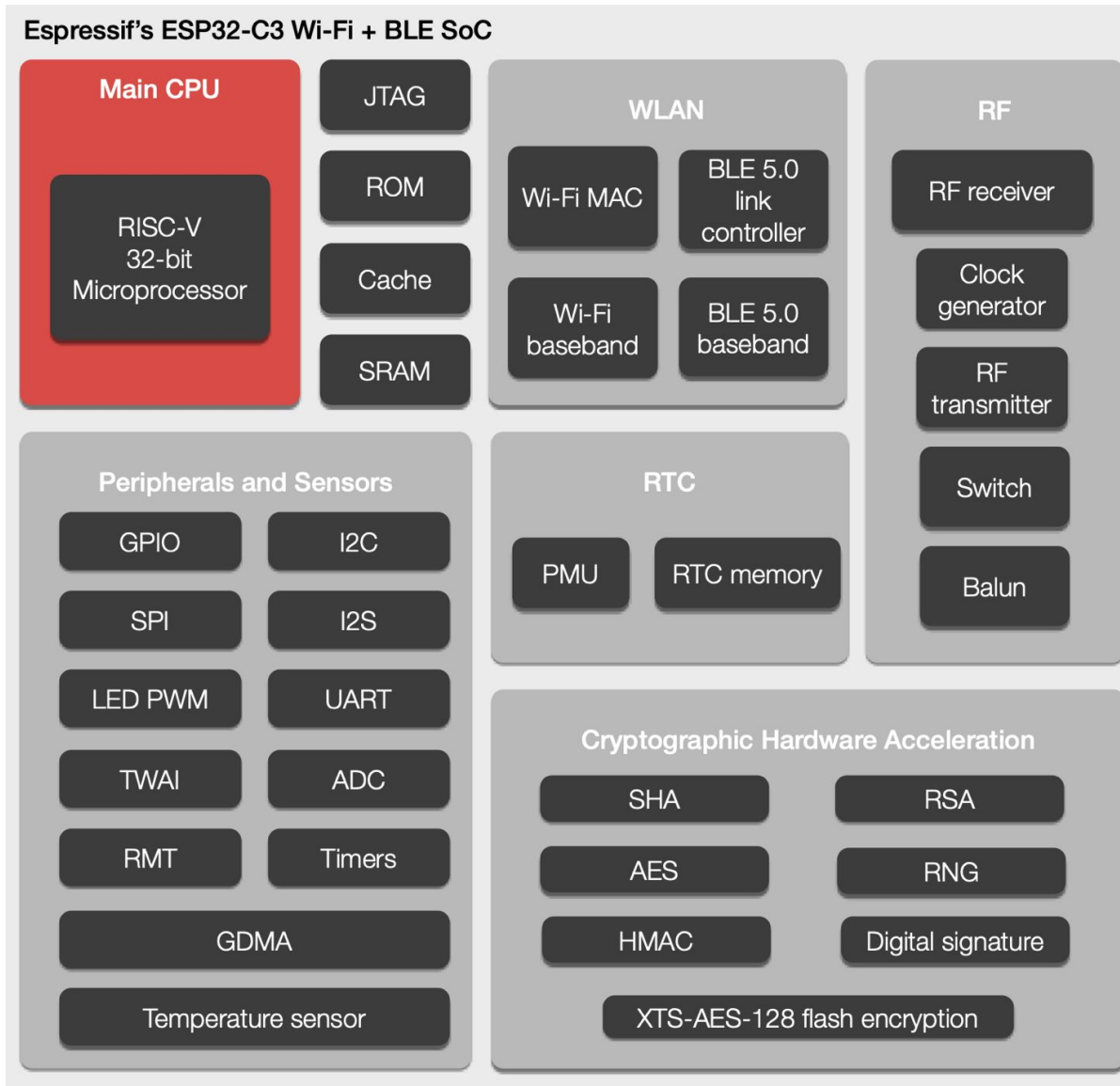
Arquitectura RISC-V del ESP32-C3

Características generales

- Características Adicionales
 - Ofrece Wi-Fi 802.11 b/g/n y Bluetooth 5 (LE) de bajo consumo.
 - Periféricos:
 - Cuenta con hasta 22 pines de E/S de propósito general (GPIO). Se pueden programar para entradas o salidas digitales, control de PWM, etc.
 - ADC de 12 bits
 - DAC de 8 bits
 - Soporta varios protocolos de comunicación a través de los pines GPIO:
 - I2C (Inter-Integrated Circuit): Para comunicación con otros chips y sensores.
 - SPI (Serial Peripheral Interface): Para comunicarse con memorias externas o periféricos de alta velocidad.
 - UART (Universal Asynchronous Receiver-Transmitter): Para la comunicación serie, a menudo utilizada con un convertidor USB a TTL para flashear el dispositivo.
 - I2S (Integrated Interchip Sound): Para la transmisión de audio.
 - Diseñado para bajo consumo, con modos de sueño profundo que reducen la corriente a microamperios.

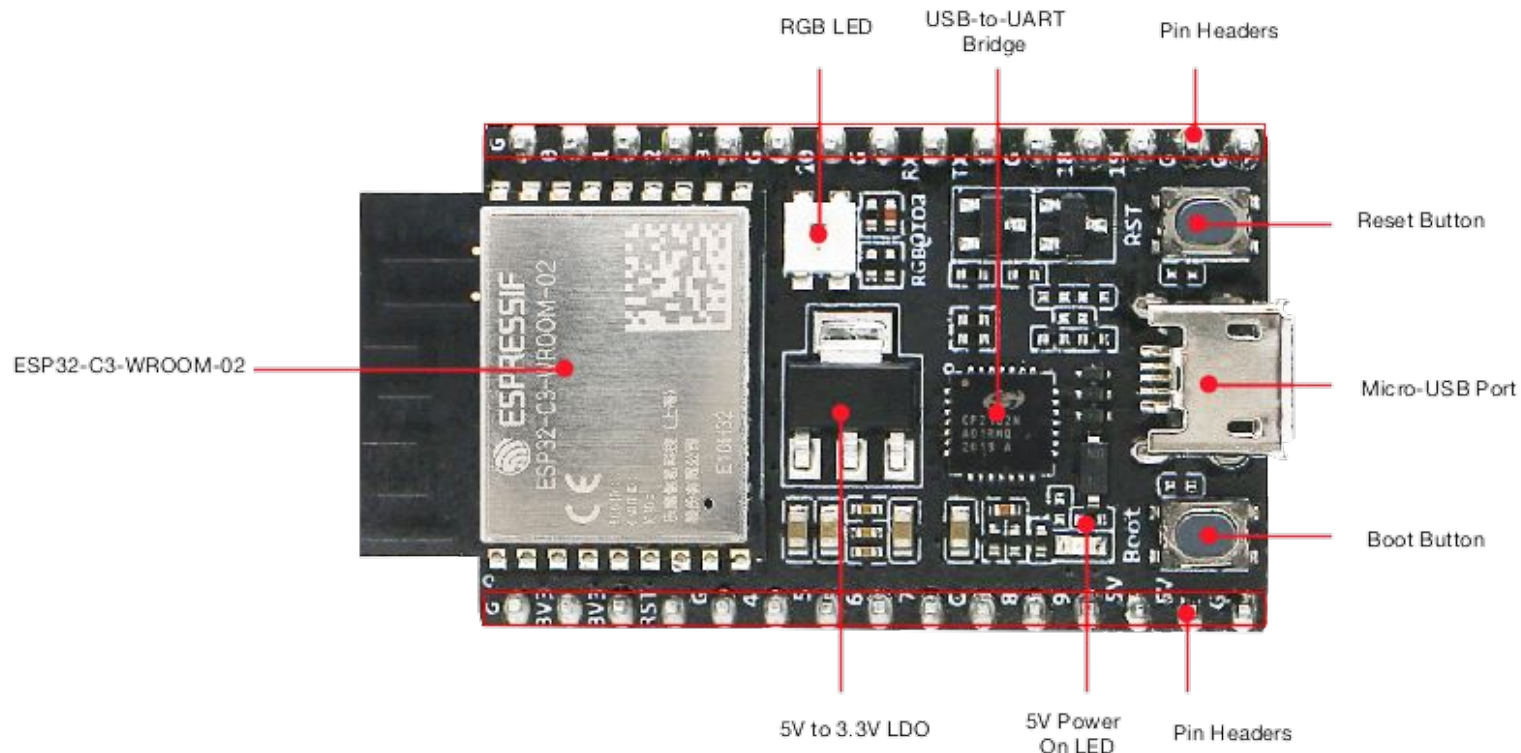
Arquitectura RISC-V del ESP32-C3

Características generales



Arquitectura RISC-V del ESP32-C3

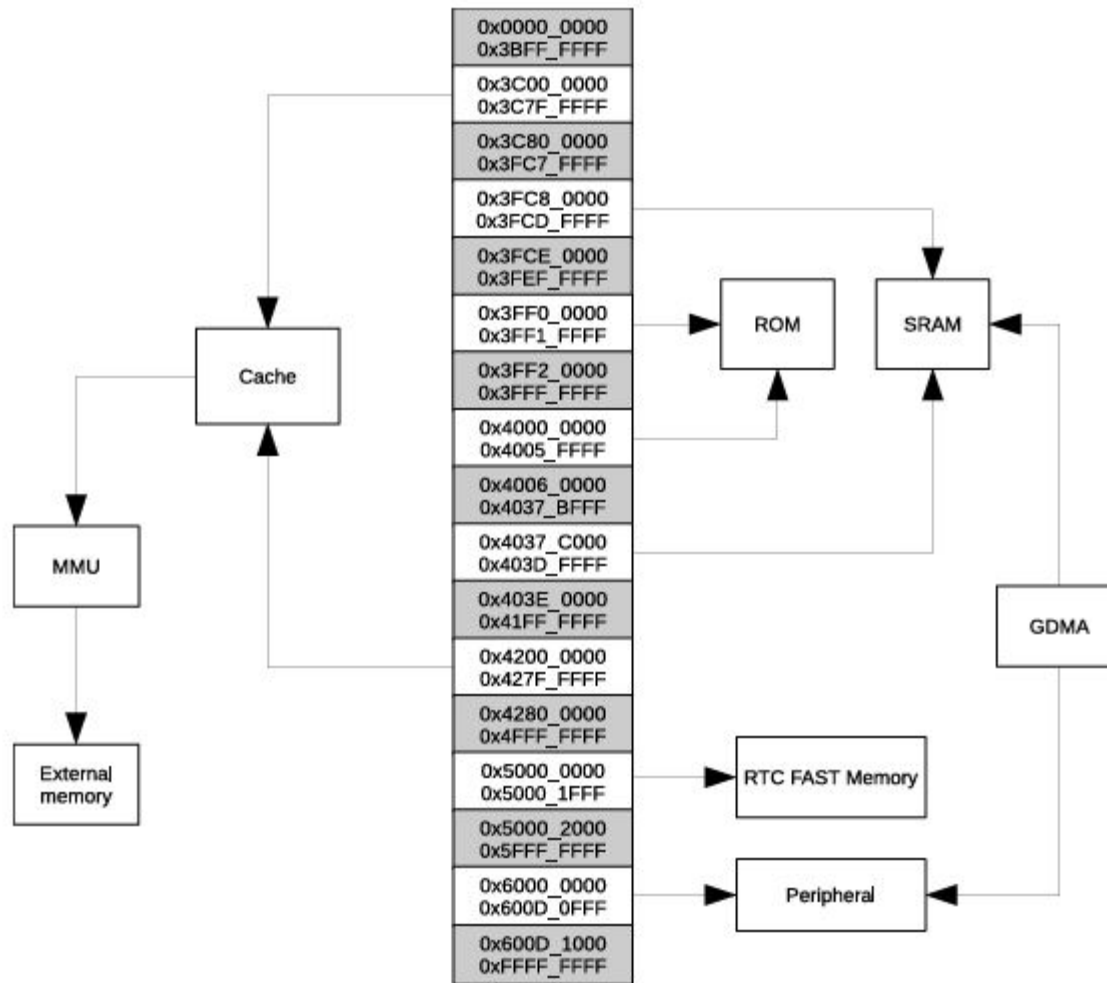
Características generales



Tomado de: <https://docs.espressif.com/projects/esp-idf/en/v5.1.2/esp32c3/hw-reference/esp32c3/user-guide-devkitc-02.html>

Arquitectura RISC-V del ESP32-C3

Memoria



Tomado de: https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf

Arquitectura RISC-V del ESP32-C3

Registros

- Registros de propósito general por la arquitectura RISC-V: X0 - X31
- Registros de control y estado (CSRs, Control and Status Registers):

Name	Description	Address	Access
Machine Trap Setup CSRs			
<code>mstatus</code>	Machine Mode Status	0x300	R/W
<code>mtvec</code>	Machine Trap Vector	0x305	R/W
Machine Trap Handling CSRs			
<code>mscratch</code>	Machine Scratch	0x340	R/W
<code>mepc</code>	Machine Trap Program Counter	0x341	R/W
<code>mcause</code>	Machine Trap Cause	0x342	R/W
<code>mtval</code>	Machine Trap Value	0x343	R/W

Tomado de: https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf

Arquitectura RISC-V del ESP32-C3

Registros

- Registros de periféricos:
 - Se usan para configurar y controlar los diversos periféricos integrados en el ESP32-C3.
 - Tipos de Periféricos:
 - GPIO (General Purpose Input/Output): 22 pines que pueden configurarse como entradas o salidas.
 - UART: Para comunicación serie.
 - SPI: Para comunicación de alta velocidad.
 - I2C: Para la comunicación con sensores y otros dispositivos I²C.
 - ADC (Convertidor Analógico-Digital): Para leer señales analógicas.
 - DAC (Convertidor Digital-Analógico): Para generar señales analógicas.
 - PWM (Modulación por Ancho de Pulso): Para controlar la duración de los pulsos.
 - TWAI (Controller Area Network): También conocido como CAN.

Arquitectura RISC-V del ESP32-C3

Registros

Name	Description	Address	Access
Configuration Registers			
GPIO_BT_SELECT_REG	GPIO bit select register	0x0000	R/W
GPIO_OUT_REG	GPIO output register	0x0004	R/W/SS
GPIO_OUT_W1TS_REG	GPIO output set register	0x0008	WT
GPIO_OUT_W1TC_REG	GPIO output clear register	0x000C	WT
GPIO_ENABLE_REG	GPIO output enable register	0x0020	R/W/SS
GPIO_ENABLE_W1TS_REG	GPIO output enable set register	0x0024	WT
GPIO_ENABLE_W1TC_REG	GPIO output enable clear register	0x0028	WT
GPIO_STRAP_REG	pin strapping register	0x0038	RO
GPIO_IN_REG	GPIO input register	0x003C	RO
GPIO_STATUS_REG	GPIO interrupt status register	0x0044	R/W/SS
GPIO_STATUS_W1TS_REG	GPIO interrupt status set register	0x0048	WT

Tomado de: https://www.espressif.com/sites/default/files/documentation/esp32-c3_technical_reference_manual_en.pdf

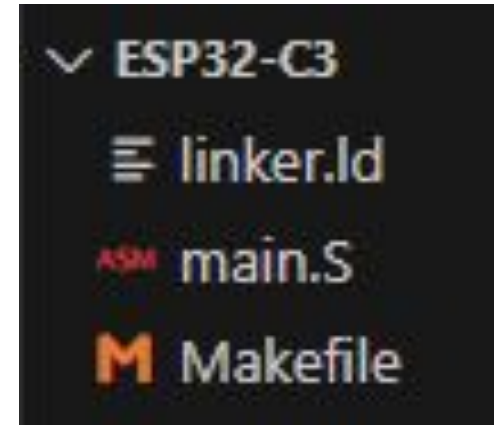
Entorno de desarrollo

Entorno de desarrollo

Configuración

proyecto/

- programa.S # Código assembler
- Makefile # Instrucciones de compilación
- linker.ld # Configuración de memoria



```
MEMORY {  
    iram (rwx) : org = 0x40380000, len = 0x20000 /* IRAM del ESP32-C3 */  
    dram (rw) : org = 0x3FC80000, len = 0x20000 /* DRAM del ESP32-C3 */  
}
```

Entorno de desarrollo

Configuración

`.global _start` Atributos de la sección (flags)

`.section .vectors, "ax"` Define una sección personalizada
`_start:`
 `j app_main` # Salto a la función principal

`.section .text`

`.section`: Directiva que define una sección de memoria

`.vectors`: Nombre de la sección para vectores de interrupción

Los vectores indican dónde saltar cuando ocurren interrupciones o excepciones

a: allocatable - La sección se cargará en memoria

x: executable - La sección contiene código ejecutable

Otros flags comunes:

w: writable - Sección escribible (datos)

r: readable - Sección readable

Entorno de desarrollo

Configuración

El ESP32-C3 tiene un vector table en la dirección 0x40380000 (IRAM).

Al bootear, la CPU:

1. Busca la dirección 0x40380000 (reset vector)
2. Ejecuta la instrucción que encuentra allí

".section .vectors, ax" le dice al ensamblador que el código que sigue debe colocarse en una zona especial de memoria donde el ESP32-C3 busca la primera instrucción al encenderse.

El código no booteara correctamente porque el reset vector no estará en la dirección esperada

Gracias